

Prof. Dr. Stefan Leutenegger

Teaching Assistants: Leonard Freißmuth and Leonard Diederich

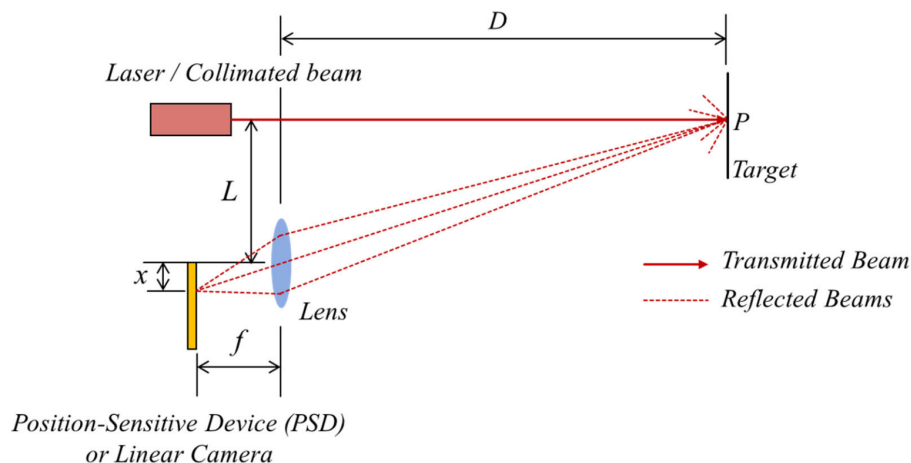
Solutions to Exercise 4: Sensor Models

Session date: 24/03/2026

Task 1: Triangulation Sensor

Consider the laser triangulation sensor shown below. A laser beam is pointed at the target and diffusely reflected through the lens onto a position-sensitive device (PSD). For the questions involving uncertainty, you may use the general, multi-dimensional formulation of the error propagation law

$$\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma}), \quad \mathbf{y} = \mathbf{f}(\mathbf{x}) \stackrel{\text{approx.}}{\sim} \mathcal{N}\left(f(\boldsymbol{\mu}), \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\boldsymbol{\mu}} \boldsymbol{\Sigma} \left. \frac{\partial \mathbf{f}}{\partial \mathbf{x}} \right|_{\boldsymbol{\mu}}^{\top}\right).$$



- i) Find the direct measurement model as introduced in the lecture

$$z_x = h_x(D) + v_x, \quad v_x \sim \mathcal{N}(0, \sigma_x^2).$$

From similar triangles, the ratio between the PSD coordinate and focal length equals the ratio between the baseline and the depth,

$$\frac{x}{f} = \frac{L}{D}.$$

Hence the direct measurement model is

$$h_x(D) = \frac{fL}{D}.$$

- ii) In practice, we are interested in the depth. Invert the model and write the distance measurement as

$$z_D = h_x^{-1}(D) + v_D = h_D(x) + v_D, \quad v_D \sim \mathcal{N}(0, \sigma_D^2).$$

Solving the previous equation for D gives

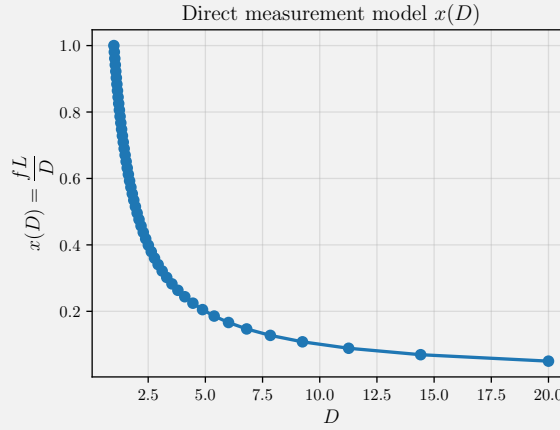
$$h_D(x) = \frac{fL}{x}.$$

- iii) Sketch the direct model, i.e. z_x as a function of D , assuming $L = 1$ and $f = 1$. Based on the sketch, discuss how the inversion affects the measurement quality (resolution, sensitivity to noise) for targets that are close and far away.

The relation is given by

$$x(D) = h_x(D) = \frac{fL}{D},$$

a monotonic hyperbola.



Interpretation:

- For **nearby targets**, a small change in depth produces a comparatively large change in the measured coordinate x . Therefore the depth resolution is good and the effect of a fixed pixel/PSD noise level is moderate.
- For **far targets**, the curve $x(D)$ becomes flatter. The same change in depth only produces a very small change in x , so the depth resolution deteriorates.

- iv) We are now interested in the uncertainty of the inverse model. Assume that the measurement $z_x \sim \mathcal{N}(x, \sigma_x^2)$ where x would be the value of the ideal, noise-free measurement. Using linear error propagation and the inversion function derived in subtask ii), find the uncertainty σ_D of the inverse model $z_D \sim \mathcal{N}(h_D(x), \sigma_D^2)$. Note that as we are dealing with a one-dimensional measurement, the formulation of the error propagation law reduces to a scalar equation.

For the scalar case, the Jacobian reduces to the derivative

$$\frac{\partial h_D(x)}{\partial x} = -\frac{fL}{x^2}.$$

The error propagation law yields

$$\sigma_D^2 = \left. \frac{\partial h_D}{\partial \mathbf{x}} \right|_{\boldsymbol{\mu}} \boldsymbol{\Sigma} \left. \frac{\partial h_D}{\partial \mathbf{x}} \right|_{\boldsymbol{\mu}}^{\top} = -\frac{fL}{x^2} \Big|_x \sigma_x^2 - \frac{fL}{x^2} \Big|_x = \frac{f^2 L^2}{x^4} \sigma_x^2.$$

Therefore,

$$\sigma_D = \frac{fL}{x^2} \sigma_x.$$

Using the nominal inverse relation

$$x = \frac{fL}{D},$$

we obtain the equivalent expression

$$\sigma_D = \frac{D^2}{fL} \sigma_x.$$

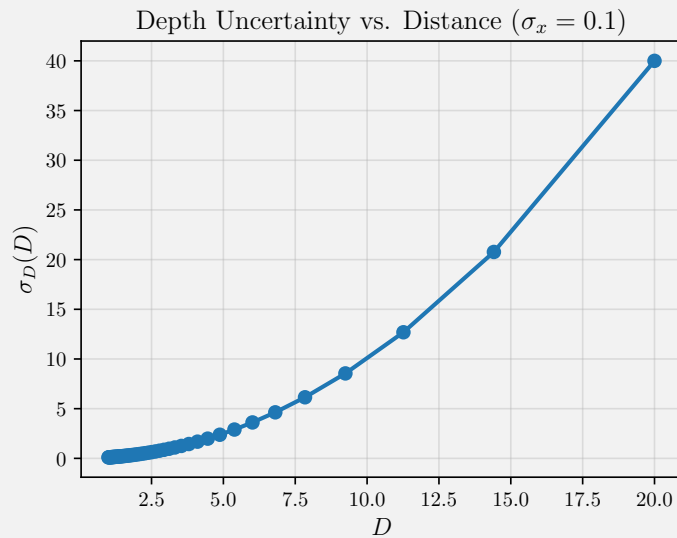
This is the usual first-order approximation obtained by linearising the inverse model around the nominal measurement.

- v) Sketch σ_D against the measured distance D assuming $\sigma_x = 0.1$. What does this equation tell you about the uncertainty for targets that are far away?

Since

$$\sigma_D(D) = \frac{D^2}{fL} \sigma_x,$$

the curve is quadratic in D and opens upward.



The uncertainty grows quadratically with distance. Hence, for far-away targets, even a small fixed uncertainty in the PSD coordinate x translates into a large uncertainty in the recovered depth D . In other words, triangulation is much more reliable at short range than at long range.

Task 2: Laser ToF (Time of Flight) Distance Sensors on Diff-Drive Robot (Python)

Consider a two-wheel differential drive mobile robot with the following states:

$$\mathbf{x}^T = [t_x, t_y, \theta]^T,$$

where t_x, t_y denote the 2D position of the robot in the world frame, and θ is the orientation of the robot frame relative to the world frame. Now the robot has three laser ToF distance sensors (index $i \in 1, 2, 3$), mounted at the robot body frame origin, pointing left ($i = 1$), forward ($i = 2$) and right ($i = 3$). Assume their measurement range is $d_{\min}$ to $d_{\max}$. While driving around in a room, it will now take distance measurements to the surrounding walls.

For installation of required packages, we provide you with a requirements.txt. You can install them using e.g. `pip install -r requirements.txt`

- i) Open the Python Script `e04_diff_drive.py` to see the set-up: the simulation will show the robot carrying out a 2D motion in a room.
- ii) Develop the ideal, noise and bias-free measurement model for the i^{th} sensor $d_i = h_i(\mathbf{x})$ by considering the intersection of the sensor ray and the wall in the world frame. Assume that the parameters a_1, a_2, a_3 of the line equation $a_1x + a_2y + a_3 = 0$ are known: these correspond to the correctly identified wall segment that the sensor is pointing at.

To derive the distance measurement, we need to find the intersection $[x^*, y^*]^T$ of the wall and the sensor direction. As for the sensor direction parameterisation, we can write

$$\begin{bmatrix} x^* \\ y^* \end{bmatrix} = \begin{bmatrix} t_x \\ t_y \end{bmatrix} + \lambda \begin{bmatrix} \cos \theta_R & -\sin \theta_R \\ \sin \theta_R & \cos \theta_R \end{bmatrix} \mathbf{e}_i, \quad (1)$$

where we find $\mathbf{e}_1 = [0, 1]^T$, $\mathbf{e}_2 = [1, 0]^T$, and $\mathbf{e}_3 = [0, -1]^T$. Simultaneously, we know the line equation, $a_1x + a_2y + a_3 = 0$. This now lets us solve the above equation for λ by plugging in (1)

$$a_1(t_x + \lambda [\cos \theta_R \quad -\sin \theta_R] \mathbf{e}_i) + a_2(t_y + \lambda [\sin \theta_R \quad \cos \theta_R] \mathbf{e}_i) + a_3 = 0,$$

therefore

$$\lambda = \frac{-a_3 - a_1 t_x - a_2 t_y}{a_1([\cos \theta_R \quad -\sin \theta_R] \mathbf{e}_i) + a_2([\sin \theta_R \quad \cos \theta_R] \mathbf{e}_i)},$$

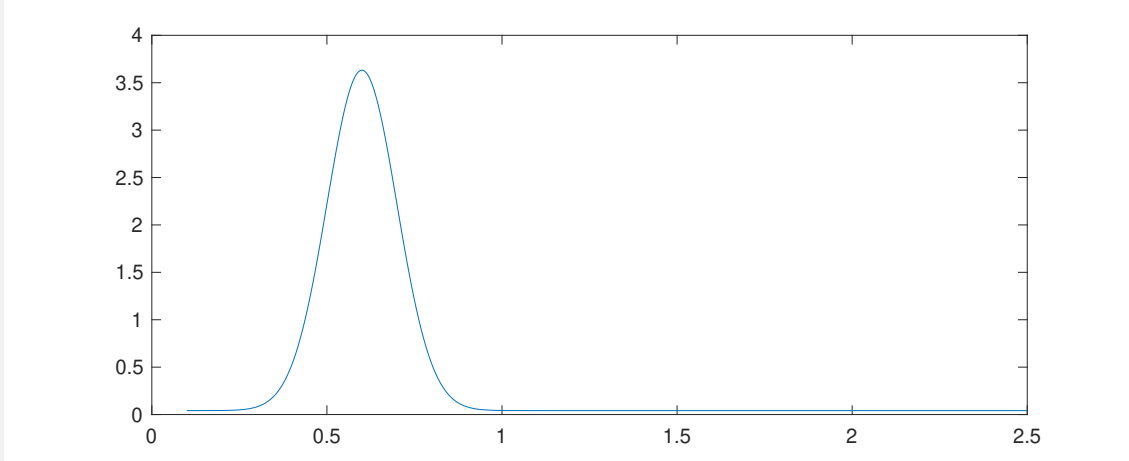
whereby we see that the denominator can be 0, if the lines are parallel, in which case there is no intersection. Furthermore, we will need to check that λ is positive, i.e. the intersection lies in front of the sensor, and that the intersection point is within the start and the end points of the line segment. With $\mathbf{x}^* = [x^*, y^*]^T$ as computed above (as a function of the state $\mathbf{x}$ and the sensor direction $\mathbf{e}_i$), we can now compute the measurement $d_i = h(\mathbf{x}^*(\mathbf{x}, \mathbf{e}_i)) = \|\mathbf{x}^* - [t_x, t_y]\|$.

- iii) Now a full probabilistic measurement model is developed on top of this. Assume that we have additive zero-mean Gaussian noise that affects the measurement with a standard deviation of σ . With a small probability of P_{outlier} , the sensor will return a nonsensical result uniformly distributed within the measurement range. Write down the full measurement model for d_i and draw the probability density function for the measurement $p(d_i)$.

We can write $d_i = (1 - X)(h_i(\mathbf{x}) + n) + Xu$, where $n \sim \mathcal{N}(0, \sigma^2)$ and $u \sim \mathcal{U}(d_{\min}, d_{\max})$ (uniformly distributed), with the auxiliary discrete random variable X , whereby $P(X = 1) = P_{\text{outlier}}$ and $P(X = 0) = 1 - P_{\text{outlier}}$. We know the probability density functions for the Normal distribution and also for u : $p(d_i) = 1/(d_{\max} - d_{\min})$. But we need to marginalise out X with the sum rule, and therefore we get:

$$p(d_i) = (1 - P_{\text{outlier}}) \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{1}{2} \frac{(d_i - h_i(\mathbf{x}))^2}{\sigma^2}\right) + \frac{P_{\text{outlier}}}{d_{\max} - d_{\min}} \mathbb{I}(d_{\min} \leq d \leq d_{\max}),$$

where $\mathbb{I}(\cdot)$ stands for the indicator function. This looks as follows:



Here, $d_{\min} = 0.1$, $d_{\max} = 2.5$, $\sigma = 0.5$, $P_{\text{outlier}} = 0.1$.

- iv) Now implement the measurement function into the Python script `e04_diff_drive.py` that is used to simulate measurements. The function should include correctly selecting the wall segment, and generate a sample from the probabilistic model above. If the distance is out of measurement range, the sensor should simply not return a measurement, i.e. an empty list. Furthermore, plot the measurements into the world frame to see, qualitatively, if things make sense. The necessary numeric values for σ , $d_{\min}$, $d_{\max}$, and P_{outlier} are already defined, but you can obviously play with them and check the effects.

See `e04_diff_drive_sol.py`.